

# Manuale Database Book Recommender

Giuseppe Antonuccio, Valerio Gazziero, Alessandro Salvini

Gennaio 2026

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                                     | <b>3</b> |
| <b>2</b> | <b>Premesse</b>                                         | <b>3</b> |
| 2.1      | Obiettivi del database . . . . .                        | 3        |
| 2.2      | Scopo del manuale database . . . . .                    | 3        |
| <b>3</b> | <b>Raccolta e analisi dei requisiti</b>                 | <b>4</b> |
| 3.1      | Metodologia di raccolta requisiti . . . . .             | 4        |
| 3.2      | Attori e scenari d'uso . . . . .                        | 4        |
| 3.2.1    | Utente non autenticato . . . . .                        | 4        |
| 3.2.2    | Utente registrato (autenticato) . . . . .               | 4        |
| 3.3      | Requisiti informativi (dati da gestire) . . . . .       | 4        |
| 3.3.1    | Libro . . . . .                                         | 4        |
| 3.3.2    | Utente registrato . . . . .                             | 5        |
| 3.3.3    | Libreria personale . . . . .                            | 5        |
| 3.4      | Valutazione . . . . .                                   | 5        |
| 3.4.1    | Suggerimento di libri correlati . . . . .               | 5        |
| <b>4</b> | <b>Requisiti funzionali e non funzionali database</b>   | <b>5</b> |
| 4.1      | Vincoli . . . . .                                       | 5        |
| 4.1.1    | Vincoli di unicità e identificazione . . . . .          | 5        |
| 4.1.2    | Vincoli di integrità referenziale . . . . .             | 5        |
| 4.1.3    | Vincoli sui domini dei valori . . . . .                 | 6        |
| 4.1.4    | Regole specifiche . . . . .                             | 6        |
| 4.2      | Requisiti non funzionali . . . . .                      | 6        |
| <b>5</b> | <b>Valutazione del carico di lavoro previsto</b>        | <b>6</b> |
| 5.1      | Volume dati iniziale . . . . .                          | 6        |
| 5.2      | Scenario e cardinalità . . . . .                        | 7        |
| 5.3      | Profilo transazionale (mix letture/scritture) . . . . . | 7        |
| 5.4      | Frequenza e concorrenza attesa . . . . .                | 8        |

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Schema scheletro (panoramica ad alto livello)</b>              | <b>8</b>  |
| 6.1       | Entità principali . . . . .                                       | 8         |
| 6.2       | Relazioni (tabelle associative) . . . . .                         | 8         |
| 6.3       | Cardinalità principali (sintesi) . . . . .                        | 8         |
| <b>7</b>  | <b>Schema concettuale (modello E-R)</b>                           | <b>9</b>  |
| 7.1       | Entità e attributi . . . . .                                      | 9         |
| 7.2       | Relazioni e vincoli di cardinalità . . . . .                      | 9         |
| 7.3       | Vincoli di dominio (concettuali) . . . . .                        | 10        |
| <b>8</b>  | <b>Schema logico (modello relazionale)</b>                        | <b>10</b> |
| <b>9</b>  | <b>Obiettivi di indicizzazione</b>                                | <b>11</b> |
| <b>10</b> | <b>Indici già presenti (automatici)</b>                           | <b>11</b> |
| <b>11</b> | <b>Indici aggiuntivi consigliati</b>                              | <b>11</b> |
| 11.1      | Ricerca testuale su titolo/autore . . . . .                       | 11        |
| 11.2      | Tabelle associative: accessi “per libro” e “per utente” . . . . . | 12        |
| 11.2.1    | LibrerieLibri . . . . .                                           | 12        |
| 11.2.2    | ValutazioniLibri . . . . .                                        | 12        |
| 11.2.3    | ConsigliLibri . . . . .                                           | 12        |
| <b>12</b> | <b>Sintesi: mapping query - indici</b>                            | <b>12</b> |
| <b>13</b> | <b>Query SQL principali</b>                                       | <b>13</b> |
| 13.1      | Utenti: registrazione e login . . . . .                           | 13        |
| 13.1.1    | Registrazione utente . . . . .                                    | 13        |
| 13.1.2    | Autenticazione . . . . .                                          | 13        |
| 13.1.3    | Recupero nome utente . . . . .                                    | 13        |
| 13.2      | Libri: consultazione e ricerca . . . . .                          | 13        |
| 13.2.1    | Ricerca per titolo . . . . .                                      | 13        |
| 13.2.2    | Ricerca per autore . . . . .                                      | 13        |
| 13.2.3    | Ricerca per autore e anno . . . . .                               | 14        |
| 13.2.4    | Recupero dettaglio libro . . . . .                                | 14        |
| 13.3      | Librerie personali . . . . .                                      | 14        |
| 13.3.1    | Creazione libreria . . . . .                                      | 14        |
| 13.3.2    | Inserimento libri in libreria (batch) . . . . .                   | 14        |
| 13.3.3    | Verifica: un libro è presente in almeno una libreria dell’utente  | 14        |
| 13.3.4    | Elenco nomi librerie di un utente . . . . .                       | 14        |
| 13.3.5    | Elenco librerie (id, nome) di un utente . . . . .                 | 14        |
| 13.3.6    | Elenco libri contenuti in una libreria . . . . .                  | 15        |
| 13.4      | Valutazioni . . . . .                                             | 15        |
| 13.4.1    | Inserimento/aggiornamento valutazione (UPSERT) . . . . .          | 15        |
| 13.4.2    | Aggregati per criterio (distribuzione 1–5 + media) . . . . .      | 15        |
| 13.4.3    | Recensioni (lista voti finali e note per libro) . . . . .         | 15        |
| 13.5      | Suggerimenti (consigli) . . . . .                                 | 16        |

|        |                                                                               |    |
|--------|-------------------------------------------------------------------------------|----|
| 13.5.1 | Conteggio suggerimenti inseriti da un utente per un libro                     | 16 |
| 13.5.2 | Sostituzione suggerimenti (delete + insert batch)                             | 16 |
| 13.5.3 | Lista titoli suggeriti da un utente per un libro                              | 16 |
| 13.5.4 | Conteggio: quante volte ogni libro è stato consigliato (per un libro origine) | 16 |

## 1 Introduzione

*BookRecommender* è un sistema client-server per la consultazione di un catalogo di libri, la gestione di librerie personali e la raccolta di valutazioni/consigli di lettura. Il cuore informativo dell'applicazione è una base di dati relazionale (PostgreSQL) che memorizza: (i) il catalogo libri importato da un dataset iniziale, (ii) i profili degli utenti registrati, (iii) le librerie personali (raccolte di libri), (iv) le valutazioni multi-criterio dei libri e (v) i suggerimenti di libri correlati inseriti dagli utenti.

## 2 Premesse

### 2.1 Obiettivi del database

Gli obiettivi principali della base di dati sono:

- garantire **integrità** e **coerenza** delle informazioni (vincoli, chiavi, referenze);
- supportare in modo efficiente le operazioni **read-intensive** di consultazione (ricerca e dettaglio libro);
- gestire operazioni **write** transazionali (registrazione, creazione librerie, inserimento/modifica valutazioni e suggerimenti);
- offrire una base solida per il calcolo di **indicatori aggregati** (ad es. medie e distribuzioni dei voti).

### 2.2 Scopo del manuale database

Questo manuale descrive:

- l'analisi dei requisiti informativi e funzionali relativi alla base di dati;
- le assunzioni di carico e dimensionamento (workload);
- lo schema logico e i vincoli implementati;
- le principali interrogazioni/operazioni SQL previste dall'applicazione.

Il documento è pensato come supporto sia alla **valutazione** del progetto sia alla **manutenzione evolutiva** del DB.

## 3 Raccolta e analisi dei requisiti

### 3.1 Metodologia di raccolta requisiti

La definizione dei requisiti per la base di dati è stata ottenuta combinando:

- **analisi della specifica di progetto** (funzioni richieste, vincoli e regole di dominio);
- **analisi dei casi d'uso** lato utente (ricerca, consultazione, valutazioni, librerie, suggerimenti);
- **ispezione dell'implementazione** (schema SQL e accessi ai dati tramite DAO), utile per allineare requisiti e comportamento effettivo.

### 3.2 Attori e scenari d'uso

#### 3.2.1 Utente non autenticato

Può consultare il catalogo e visualizzare le informazioni di dettaglio, inclusi prospetti aggregati delle valutazioni e suggerimenti disponibili.

#### 3.2.2 Utente registrato (autenticato)

Oltre alle funzionalità di consultazione, può:

- creare e gestire librerie personali;
- inserire o aggiornare valutazioni multi-criterio dei libri;
- associare fino a un numero massimo di suggerimenti di libri correlati per un libro dato.

### 3.3 Requisiti informativi (dati da gestire)

Di seguito i principali concetti informativi che il DB deve rappresentare.

#### 3.3.1 Libro

Per ogni libro è necessario memorizzare almeno:

- titolo, autore;
- anno di pubblicazione (se disponibile);
- genere/categoria;
- descrizione testuale.

Il catalogo viene inizialmente popolato tramite import da un dataset CSV.

### 3.3.2 Utente registrato

Per ogni utente registrato devono essere memorizzati:

- dati anagrafici minimi (nome, cognome);
- identificativi univoci (codice fiscale, email, userid);
- credenziali (password in forma di **hash**, mai in chiaro).

### 3.3.3 Libreria personale

Ogni utente può creare una o più librerie, ognuna identificata da un nome (univoco **per utente**) e associata a un insieme di libri.

## 3.4 Valutazione

Un utente registrato può associare una valutazione a un libro, composta da più criteri su scala discreta, più una nota testuale opzionale. La valutazione è univoca per coppia (**utente**, **libro**) (l'utente può eventualmente aggiornare la propria valutazione).

### 3.4.1 Suggerimento di libri correlati

Un utente registrato può associare al libro “corrente” un insieme limitato di libri “suggeriti” come correlati. Il vincolo di massimo numero di suggerimenti per (**utente**, **libro**) è una regola di business.

## 4 Requisiti funzionali e non funzionali database

### 4.1 Vincoli

I principali vincoli, necessari per evitare inconsistenze e duplicazioni, sono:

#### 4.1.1 Vincoli di unicità e identificazione

- un utente è identificato da una chiave surrogata **id**;
- `codice_fiscale`, `email` e `userid` sono **univoci**;
- una libreria è univoca rispetto alla coppia (**utente**, **nome**).

#### 4.1.2 Vincoli di integrità referenziale

- una libreria appartiene a un utente registrato;
- l'associazione libro-libreria e le valutazioni fanno riferimento a libri esistenti;

- in caso di cancellazione utente, i dati dipendenti (librerie, valutazioni, suggerimenti) devono essere gestiti coerentemente (es. cancellazione a cascata).

#### 4.1.3 Vincoli sui domini dei valori

- ciascun criterio di valutazione è definito su una scala discreta (ad es. 1–5);
- la nota testuale della valutazione è opzionale e limitata in lunghezza.

#### 4.1.4 Regole specifiche

- un utente può esprimere **al massimo una valutazione** per libro (aggiornabile);
- un utente può associare **al massimo 3 suggerimenti** per ciascun libro (vincolo logico di dominio).

### 4.2 Requisiti non funzionali

- **Affidabilità e integrità:** uso di vincoli e transazioni per evitare dati orfani o duplicati.
- **Prestazioni:** ottimizzazione per ricerche testuali (titolo/autore) e consultazione aggregati.
- **Scalabilità:** supporto a crescita del numero di utenti e di interazioni (valutazioni, librerie, suggerimenti).
- **Sicurezza:** persistenza di password esclusivamente come hash; limitazione dell'accesso al DB tramite credenziali.
- **Portabilità:** adozione di PostgreSQL e SQL standard ove possibile; accesso via JDBC.

## 5 Valutazione del carico di lavoro previsto

Questa sezione stima il carico atteso per dimensionare in modo ragionevole tabelle, indici e strategie di accesso. Le stime sono espresse come **ordine di grandezza** e possono essere rifinite in base al contesto di deployment (laboratorio, demo, uso reale).

### 5.1 Volume dati iniziale

Il catalogo viene importato da un file CSV di circa **97.000 righe** (ordine di grandezza). Pertanto, si prevede:

- **Libri:**  $\sim 10^5$  record.

## 5.2 Scenario e cardinalità

Si assumono scenari compatibili con un progetto didattico (uso in classe/lab) con possibile estensione a piccoli gruppi.

| Oggetto                          | Stima           | Nota                |
|----------------------------------|-----------------|---------------------|
| Utenti registrati                | $10^2$ – $10^3$ | crescita moderata   |
| Librerie per utente              | 3–10            | librerie tematiche  |
| Libri per libreria               | 20–200          | dipende dall'uso    |
| Valutazioni per utente           | 20–200          | nel tempo           |
| Suggerimenti per (utente, libro) | 0–3             | vincolo di business |

Table 1: Assunzioni per il dimensionamento (ordine di grandezza).

Da tali ipotesi discendono stime indicative sulle cardinalità delle tabelle principali:

- **Librerie:** fino a  $\sim 10^4$  (in caso di  $10^3$  utenti con 10 librerie ciascuno).
- **Librerie\_Libri:** fino a  $\sim 10^6$  associazioni (scenario alto: molte librerie popolose).
- **ValutazioniLibri:** fino a  $\sim 10^5$ – $2 \cdot 10^5$  (se ogni utente valuta decine/centinaia di libri).
- **ConsigliLibri:** fino a  $\sim 3 \cdot$  ValutazioniLibri nel caso limite (tutti inseriscono 3 suggerimenti).

## 5.3 Profilo transazionale (mix letture/scritture)

Il carico è tipicamente **sbilanciato sulle letture**:

- **Operazioni di lettura (prevalenti):** ricerca libro, apertura dettaglio, consultazione aggregati, consultazione suggerimenti.
- **Operazioni di scrittura (meno frequenti):** registrazione, creazione librerie, aggiunta libri a librerie, inserimento/aggiornamento valutazioni, inserimento suggerimenti.

Una stima ragionevole del mix è:

$$\text{Read : Write} \approx 90 : 10$$

con possibili picchi di scrittura durante sessioni guidate (es. laboratorio in cui molti studenti inseriscono valutazioni nello stesso intervallo).

## 5.4 Frequenza e concorrenza attesa

In un contesto di laboratorio si può assumere:

- concorrenza: 10–30 client contemporanei;
- per utente attivo: 5–20 ricerche e 5–30 consultazioni dettaglio per sessione;
- scritture: 0–5 operazioni significative per sessione (valutazioni/librerie/suggerimenti).

Schema scheletro, schema concettuale e schema logico

## 6 Schema scheletro (panoramica ad alto livello)

Lo schema del database è costruito attorno a tre entità principali (**Utente**, **Libro**, **Libreria**) e tre tabelle associative per modellare relazioni multi-a-molti e relazioni con attributi.

### 6.1 Entità principali

- **UtentiRegistrati**: anagrafica e credenziali degli utenti registrati.
- **Libri**: catalogo libri importato da dataset, consultabile e valutabile.
- **Librerie**: collezioni personali create dagli utenti (un utente può avere più librerie).

### 6.2 Relazioni (tabelle associative)

- **Librerie\_Libri**: associazione tra librerie e libri (multi-a-molti).
- **ValutazioniLibri**: associazione tra utente e libro con attributi di valutazione (una valutazione per coppia utente-libro).
- **ConsigliLibri**: associazione che memorizza i libri suggeriti come correlati, riferita a un utente e a un libro di partenza.

### 6.3 Cardinalità principali (sintesi)

- Utente **1:N** Libreria
- Libreria **N:M** Libro (tramite Librerie\_Libri)
- Utente **N:M** Libro (tramite ValutazioniLibri, con vincolo di unicità sulla coppia)
- Utente **1:N** Consiglio (per ciascun libro di partenza; vincolo logico: max 3 suggerimenti per (utente, libro))



## 7 Schema concettuale (modello E-R)

### 7.1 Entità e attributi

| Entità           | Identificatore | Attributi principali                                        |
|------------------|----------------|-------------------------------------------------------------|
| UtenteRegistrato | id             | nome, cognome, codice_fiscale, email, userid, password_hash |
| Libro            | id             | titolo, autore, anno, genere, descrizione                   |
| Libreria         | id             | nome                                                        |

Table 2: Entità del modello concettuale.

### 7.2 Relazioni e vincoli di cardinalità

- **Possiede** (UtenteRegistrato–Libreria):
  - cardinalità: UtenteRegistrato (1) – Libreria (N);
  - vincolo: il nome della libreria è univoco **per utente**.
- **Contiene** (Libreria–Libro):
  - cardinalità: Libreria (N) – Libro (M);
  - interpretazione: una libreria contiene più libri, un libro può comparire in più librerie (anche di utenti diversi).
- **Valuta** (UtenteRegistrato–Libro) con attributi:
  - cardinalità concettuale: N:M;
  - vincolo: **una sola valutazione per coppia (utente, libro)**, aggiornabile;
  - attributi della relazione: **stile, contenuto, gradevolezza, originalita, edizione, note**;
  - attributo derivato: **voto\_finale** calcolato dai criteri.
- **Consiglia** (relazione ternaria) (UtenteRegistrato, Libro\_origine, Libro\_suggerito):
  - l'utente propone uno o più libri suggeriti a partire da un libro di origine;
  - vincolo di business: **massimo 3 libri suggeriti per ciascuna coppia (utente, libro\_origine)**;
  - vincolo di unicità naturale: non ha senso ripetere lo stesso libro suggerito per la stessa coppia (utente, libro\_origine).

### 7.3 Vincoli di dominio (concettuali)

- I punteggi dei criteri di valutazione appartengono a una scala discreta 1–5.
- La nota associata alla valutazione è opzionale e di lunghezza limitata.

## 8 Schema logico (modello relazionale)

Lo schema logico traduce il modello concettuale in tabelle relazionali con chiavi primarie, chiavi esterne e vincoli. Si riportano al seguito le relazioni:

- **UtentiRegistrati**(id, nome, cognome, codice\_fiscale, email, userid, password\_hash)
  - vincoli: `codice_fiscale`, `email`, `userid` univoci.
- **Libri**(id, titolo, autore, anno, genere, descrizione)
- **Librerie**(id, utente\_id, nome)
  - `utente_id` → `UtentiRegistrati(id)` (FK, ON DELETE CASCADE)
  - vincolo: `UNIQUE(utente_id, nome)`.
- **Librerie\_Libri**(libreria\_id, libro\_id)
  - `libreria_id` → `Librerie(id)` (FK, ON DELETE CASCADE)
  - `libro_id` → `Libri(id)` (FK, ON DELETE CASCADE)
  - implementa la relazione multi-a-molti *Contiene*.
- **ValutazioniLibri**(utente\_id, libro\_id, stile, contenuto, gradevolezza, originalita, edizione, voto\_finale, note)
  - `utente_id` → `UtentiRegistrati(id)` (FK, ON DELETE CASCADE)
  - `libro_id` → `Libri(id)` (FK, ON DELETE CASCADE)
  - vincoli di dominio: `CHECK(x BETWEEN 1 AND 5)` per ciascun criterio;
  - vincolo di unicità: `PK(utente_id, libro_id)` garantisce una sola valutazione per coppia;
  - `voto_finale` è un attributo **generato** (colonna calcolata e memorizzata) come media dei cinque criteri, con arrotondamento a 2 decimali.
- **ConsigliLibri**(utente\_id, libro\_id, suggerito\_libro\_id)
  - `utente_id` → `UtentiRegistrati(id)` (FK, ON DELETE CASCADE)
  - `libro_id` → `Libri(id)` (FK, ON DELETE CASCADE)
  - `suggerito_libro_id` → `Libri(id)` (FK, ON DELETE CASCADE)
  - `PK(utente_id, libro_id, suggerito_libro_id)` evita duplicati dello stesso suggerimento.

Indici

## 9 Obiettivi di indicizzazione

Il carico dell'applicazione è prevalentemente **read-intensive**: molte ricerche/consultazioni e un numero più contenuto di scritture (valutazioni, librerie, suggerimenti). Gli indici servono a:

- accelerare la ricerca dei libri (titolo/autore);
- rendere efficienti join e filtri sulle tabelle associative;
- velocizzare aggregazioni e consultazioni per `libro_id`.

## 10 Indici già presenti (automatici)

PostgreSQL crea automaticamente indici **B-Tree** per:

- **PRIMARY KEY** di tutte le tabelle;
- **UNIQUE** su `UtentiRegistrati(codice_fiscale)`, `UtentiRegistrati(email)`, `UtentiRegistrati(userid)`;
- **UNIQUE(utente\_id, nome)** su `Librerie`, utile anche per le query che filtrano per `utente_id` e ordinano per `nome`.

## 11 Indici aggiuntivi consigliati

Lo `schema.sql` definisce correttamente chiavi e vincoli, ma alcune query filtrano su colonne che non sono coperte da un indice “in testa” (es. `WHERE libro_id=?` su tabelle con PK diversa). Per questi casi sono consigliati indici aggiuntivi.

### 11.1 Ricerca testuale su titolo/autore

Le query di ricerca usano:

```
WHERE LOWER(titolo) LIKE LOWER(%q%) e WHERE LOWER(autore) LIKE LOWER(%q%)
```

Con il pattern `%q%` un semplice B-Tree spesso non basta (non sfrutta bene i wildcard iniziali). In PostgreSQL, per rendere efficiente questo tipo di ricerca è consigliabile usare l'estensione `pg_trgm` e indici GIN.

#### Opzione consigliata (PostgreSQL + trigrammi)

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

```
CREATE INDEX IF NOT EXISTS idx_libri_titolo_trgm  
ON Libri USING gin (LOWER(titolo) gin_trgm_ops);
```

```
CREATE INDEX IF NOT EXISTS idx_libri_autore_trgm  
ON Libri USING gin (LOWER(autore) gin_trgm_ops);
```

**Opzione minimale (B-Tree, utile solo per ricerche prefix)** Se l'applicazione venisse modificata per usare ricerche del tipo `q%` (prefisso), allora un B-Tree su espressione sarebbe sufficiente:

```
CREATE INDEX IF NOT EXISTS idx_libri_titolo_lower ON Libri (LOWER(titolo));
CREATE INDEX IF NOT EXISTS idx_libri_autore_lower ON Libri (LOWER-autore));
```

## 11.2 Tabelle associative: accessi “per libro” e “per utente”

### 11.2.1 LibrerieLibri

La PK (`libreria_id`, `libro_id`) supporta bene: `WHERE libreria_id=?` (lista libri di una libreria). Per la ricerca inversa `WHERE libro_id=?` (utile nei join per verifiche di appartenenza), conviene un indice dedicato:

```
CREATE INDEX IF NOT EXISTS idx_librerie_libri_libro
ON LibrerieLibri (libro_id);
```

### 11.2.2 ValutazioniLibri

La PK (`utente_id`, `libro_id`) non è ideale per query che filtrano solo per `libro_id` (aggregati e lista recensioni di un libro). Si consiglia:

```
CREATE INDEX IF NOT EXISTS idx_valutazioni_libro
ON ValutazioniLibri (libro_id);
```

Opzionalmente (PostgreSQL), per favorire *index-only scan* nelle schermate di dettaglio:

```
CREATE INDEX IF NOT EXISTS idx_valutazioni_libro_cover
ON ValutazioniLibri (libro_id)
INCLUDE (utente_id, voto_finale, note, stile, contenuto, gradevolezza, originalita, ediz);
```

### 11.2.3 ConsigliLibri

La PK (`utente_id`, `libro_id`, `suggerito_libro_id`) copre già bene: `WHERE utente_id=? AND libro_id=?` (conteggio, delete, lista suggeriti per utente-libro). Per la query di conteggio aggregata per libro (`WHERE libro_id=? GROUP BY suggerito_libro_id`) è utile un indice che inizi con `libro_id`:

```
CREATE INDEX IF NOT EXISTS idx_consigli_libro_suggerito
ON ConsigliLibri (libro_id, suggerito_libro_id);
```

## 12 Sintesi: mapping query - indici

- Ricerca per titolo/autore (`LIKE %q%`) → `idx.libri.*_trgm`.
- Libri in una libreria (`WHERE libreria_id=?`) → PK `LibrerieLibri`.

- Verifica libro nelle librerie dell'utente (JOIN ... WHERE utente\_id=? AND libro\_id=?) → UNIQUE(utente\_id,nome) su Librerie + idx.librerie\_libri\_libro.
- Aggregati/recensioni per libro (WHERE libro\_id=?) → idx.valutazioni\_libro.
- Conteggi consigli per libro (GROUP BY suggerito\_libro\_id) → idx.consigli\_libro\_suggerito.

## 13 Query SQL principali

### 13.1 Utenti: registrazione e login

#### 13.1.1 Registrazione utente

Inserimento di un nuovo utente e ritorno dell'identificatore assegnato.

```
INSERT INTO UtentiRegistrati(
    nome, cognome, codice_fiscale, email, userid, password_hash
) VALUES (?, ?, ?, ?, ?, ?) RETURNING id;
```

#### 13.1.2 Autenticazione

Ricerca dell'utente per userid oppure email e verifica password\_hash.

```
SELECT id
FROM UtentiRegistrati
WHERE (userid=? OR email=?) AND password_hash=;
```

#### 13.1.3 Recupero nome utente

```
SELECT nome
FROM UtentiRegistrati
WHERE id=;
```

### 13.2 Libri: consultazione e ricerca

#### 13.2.1 Ricerca per titolo

```
SELECT *
FROM Libri
WHERE LOWER(titolo) LIKE LOWER(?)
ORDER BY titolo;
```

Parametro: %q%.

#### 13.2.2 Ricerca per autore

```
SELECT *
FROM Libri
WHERE LOWER-autore) LIKE LOWER(?)
ORDER BY titolo;
```

### 13.2.3 Ricerca per autore e anno

```
SELECT *
FROM Libri
WHERE LOWER-autore) LIKE LOWER(?) AND anno=?
ORDER BY titolo;
```

### 13.2.4 Recupero dettaglio libro

```
SELECT *
FROM Libri
WHERE id=?;
```

## 13.3 Librerie personali

### 13.3.1 Creazione libreria

```
INSERT INTO Librerie(utente_id, nome)
VALUES (?,?) RETURNING id;
```

### 13.3.2 Inserimento libri in libreria (batch)

L'inserimento evita duplicati grazie a ON CONFLICT DO NOTHING.

```
INSERT INTO Librerie_Libri(libreria_id, libro_id)
VALUES (?,?) ON CONFLICT DO NOTHING;
```

### 13.3.3 Verifica: un libro è presente in almeno una libreria dell'utente

```
SELECT 1
FROM Librerie l
JOIN Librerie_Libri ll ON l.id=ll.libreria_id
WHERE l.utente_id=? AND ll.libro_id=?;
```

### 13.3.4 Elenco nomi librerie di un utente

```
SELECT nome
FROM Librerie
WHERE utente_id=?
ORDER BY nome;
```

### 13.3.5 Elenco librerie (id, nome) di un utente

```
SELECT id, nome
FROM Librerie
WHERE utente_id=?
ORDER BY nome;
```

### 13.3.6 Elenco libri contenuti in una libreria

```
SELECT libro_id
FROM LibrerieLibri
WHERE libreria_id=?
ORDER BY libro_id;
```

## 13.4 Valutazioni

### 13.4.1 Inserimento/aggiornamento valutazione (UPSERT)

La valutazione è univoca per (utente\_id, libro\_id).

```
INSERT INTO ValutazioniLibri(
    utente_id, libro_id, stile, contenuto, gradevolezza, originalita, edizione, note
) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
ON CONFLICT(utente_id, libro_id) DO UPDATE SET
    stile=EXCLUDED.stile,
    contenuto=EXCLUDED.contenuto,
    gradevolezza=EXCLUDED.gradevolezza,
    originalita=EXCLUDED.originalita,
    edizione=EXCLUDED.edizione,
    note=EXCLUDED.note;
```

Nota: voto\_finale è una colonna **generata** nello schema e si aggiorna automaticamente.

### 13.4.2 Aggregati per criterio (distribuzione 1-5 + media)

Per ogni campo tra stile, contenuto, gradevolezza, originalita, edizione, voto\_finale viene eseguita la seguente query (con %1\$s sostituito dal nome campo):

```
SELECT
    COUNT(*) FILTER (WHERE %1$s=1) c1,
    COUNT(*) FILTER (WHERE %1$s=2) c2,
    COUNT(*) FILTER (WHERE %1$s=3) c3,
    COUNT(*) FILTER (WHERE %1$s=4) c4,
    COUNT(*) FILTER (WHERE %1$s=5) c5,
    ROUND(AVG(%1$s),2) media
FROM ValutazioniLibri
WHERE libro_id=?;
```

### 13.4.3 Recensioni (lista voti finali e note per libro)

```
SELECT utente_id, voto_finale, note
FROM ValutazioniLibri
WHERE libro_id=?
ORDER BY utente_id;
```

## 13.5 Suggerimenti (consigli)

### 13.5.1 Conteggio suggerimenti inseriti da un utente per un libro

Utile per applicare la regola “max 3” lato applicazione.

```
SELECT COUNT(*)
FROM ConsigliLibri
WHERE utente_id=? AND libro_id=?;
```

### 13.5.2 Sostituzione suggerimenti (delete + insert batch)

Prima si eliminano i suggerimenti esistenti per quella coppia (utente, libro), poi si inseriscono i nuovi (max 3).

```
DELETE FROM ConsigliLibri
WHERE utente_id=? AND libro_id=?;

INSERT INTO ConsigliLibri(utente_id, libro_id, suggerito_libro_id)
VALUES (?, ?, ?);
```

### 13.5.3 Lista titoli suggeriti da un utente per un libro

```
SELECT l.titolo
FROM ConsigliLibri c
JOIN Libri l ON c.suggerito_libro_id = l.id
WHERE c.utente_id=? AND c.libro_id=?
ORDER BY l.titolo;
```

### 13.5.4 Conteggio: quante volte ogni libro è stato consigliato (per un libro origine)

Questa query supporta la schermata di dettaglio, mostrando i suggerimenti più frequenti.

```
SELECT suggerito_libro_id, COUNT(*) n
FROM ConsigliLibri
WHERE libro_id=?
GROUP BY suggerito_libro_id
ORDER BY n DESC;
```